

1. Spatial/Temporal Locality**08A1**

局部性可以体现在**空间**上，也可体现在**时间**上。二者有何联系与区别？

2. Splay-1**08A1**

讲义指出，若将关键码集存储为伸展树，并**单调且周而复始**地查找，则均摊时间成本将高达 $\Omega(n)$ 。

如果改为按某一**随机**次序，**周而复始**地查找呢？

3. Splay-1**08A1**

设针对一个规模为 r 的关键码**子集**，同样**单调且周期性**地对伸展树做查找。试证明：

- 在第一轮查找之后，接下来的每一次查找过程中所造访的节点，**都来自**上述子集（而与其余节点**无关**）；
- 单次查找的均摊时间成本为 $\mathcal{O}(r)$ 。

4. Splay-2 = Folding**08A2**

Tarjan的**双层**伸展法之所以能够破解**逐层**伸展法的最坏情况，乃是因为双层伸展能产生一种**折叠**效果。

试借助演示工具观察和确认这种**效果**，并理解两种伸展策略的本质**差异**。

5. Height Difference**08A3**

伸展树的插入、删除操作，都可能引起树高的变化。

- 树高何时会**降低**？何时反而会**增加**？
- 如果是降低，降低的比率**最大**可能达到多少？
- 分别针对插入、删除，**构造**出可以达到上述比率的实例。

6. Splay::remove()**08A3**

出于简捷考虑，本节实现的伸展树删除算法并不**对称**。试从以下方面加以完善：

- 当左子树为空时，也应直接取右子树作为**余树**；
- 当左、右子树都非空时，也应以**50%**的概率，在左子树中对目标做二次查找（在其中的最大者伸展至根后，亦必然没有右子树，于是可将右子树作为其右子树）。

7. Amortization**08A4**

本节讲义针对伸展树性能的均摊分析，讨论了三种典型的情况。其中第一种情况（被伸展节点的原深度为奇数时，最终需要**单旋**一次）幸亏在每次伸展过程中至多出现**一次**（最后一步）。

- 试设想一下，如果这种情况可能出现**多次**，均摊分析的结论是否依然成立？为什么？
- 从势能的角度来看，采用**逐层**伸展策略的BST，为何单次操作的均摊时间复杂度**不能**控制在 $\mathcal{O}(\log n)$ ？

8. Tightness Of Upper Bounds**08A4**

本节已证明：在任一节点 v 的伸展过程中，它每上升**两层**（从第 $i-1$ 个状态转入第 i 个状态）所需的均摊时间成本总是满足：

$$A_i = T_i + \Delta\Phi_i = \mathcal{O}(\Delta\text{rank}_i(v)) = \mathcal{O}(\text{rank}_i(v) - \text{rank}_{i-1}(v))$$

特别地，当 “ $v == p \rightarrow lc \ \&\& \ p == g \rightarrow rc$ ” 或 “ $v == p \rightarrow rc \ \&\& \ p == g \rightarrow lc$ ” 时，必满足：

$$A_i < 2 \times \Delta\text{rank}_i(v)$$

a) 该估计式在什么情况下**最紧**? 试画出此时该树在此处的**局部结构**的示意图, 并**解释**为何最紧。

b) 对任一**偶数** h , 试构造一棵高度为 h 的伸展树, 并指定其中的一个节点 v , 使得:

$$\lim_{h \rightarrow +\infty} A(h)/\log(n(h)) = 1/2$$

其中, $A(h)$ 为将 v 伸展至根所需的均摊时间成本, $n(h)$ 为该树的规模。

试画出该树**整体结构**的示意图, 标注 v 的位置以及其它必要的信息; 同时推导出 $n(h)$ 的表达式。

9. Tightness Of Upper Bounds

08A4

本节已证明: 在任一节点 v 的伸展过程中, 它每上升**两层** (从第 $i-1$ 个状态转入第 i 个状态) 所需的均摊时间成本总是满足:

$$A_i = T_i + \Delta\Phi_i = \mathcal{O}(\Delta\text{rank}_i(v)) = \mathcal{O}(\text{rank}_i(v) - \text{rank}_{i-1}(v))$$

特别地, 当 “ $v == p \rightarrow \text{lc} \ \&\& \ p == g \rightarrow \text{lc}$ ” 或 “ $v == p \rightarrow \text{rc} \ \&\& \ p == g \rightarrow \text{rc}$ ” 时, 必满足:

$$A_i < 3 \times \Delta\text{rank}_i(v)$$

a) 该估计式在什么情况下**最紧**? 试画出此时该树在此处的**局部结构**的示意图, 并**解释**为何最紧。

b) 对任一**偶数** h , 试构造一棵高度为 h 的伸展树, 并指定其中的一个节点 v , 使得:

$$\lim_{h \rightarrow +\infty} A(h)/\log(n(h)) = 1/3$$

其中, $A(h)$ 为将 v 伸展至根所需的均摊时间成本, $n(h)$ 为该树的规模。

试画出该树**整体结构**的示意图, 标注 v 的位置以及其它必要的信息; 同时推导出 $n(h)$ 的表达式。

10. Cache

08B1

查阅资料了解Cache的**原理与机制**, 了解在典型计算机系统中Cache的**组成**, 比如:

R. Bryant & D. O'Hallaron, *Computer Systems: A Programmer's Perspective* 之第六章

11. k-Shift

08B1

讲义中针对k-Shift问题给出了几种解法, 并从**发挥**系统缓存作用的角度做了分析比较。试在示例代码中找到对应的代码 (或自己重写), 针对不同规模的数据做一性能测试, **验证或推翻**课上给出的结论。

12. k-Shift

08B1

如果放宽条件, 允许使用 $\mathcal{O}(d)$ 的辅助空间 ($d = \gcd(n, k)$), 试设计一个运行时间为 $\mathcal{O}(n)$ 的算法 $\text{shift}(A[], n, k)$, 且数据的方法符合 “**Stride-1 Reference Pattern**”, 从而能充分发挥缓存的作用。

13. setvbuf()

08B2

C语言在<studio.h>库中提供的**setvbuf()**函数, 可以帮助你定制缓冲区的大小, 以及缓冲的方式。

试查阅相关资料, 并通过编程、运行、统计, 对不同缓冲设置的效果做一对比分析。

14. #Keys vs. #External Nodes

08B3

试证明: 无论阶次高低, 无论关键码多少, 任何B-树中的**关键码**总是恰好比**外部节点**少一个。

15. Size Of Child Vector

08B3

示例代码中定义的BTNode里, child向量、key向量的容量都取作 m 。而按照B-树的约定, 每个节点中的key

不会超过 $m-1$ 个，是否因此可在定义中将该向量的容量减少一个单位？

16. BTreeNode Construction

08B3

按照示例代码，新生的BTreeNode必然**父亲**为nullptr，**高度**默认为1，没有**关键码**但只有一个**孩子**。

试联系后面的BTree::insert()算法，确认如此实现已经**足够**。

17. B-Tree Height

08B4

与BST一样，B-树的**查找**性能也决定于其**高度**，讲义中则针对其可能的**最小**、**最大**高度做了严格界定。

- 试由此反推出，任一特定高度的B-树中，最多、最少含有多少个**关键码**？
- 随着高度的增大，这两个数值之间的**跨度**将以什么速度增长？
- 这样的增长速度意味着什么？

18. B-Tree Traversal

08B4

既然B-树乃是二叉树的一种等价形式，后者的**先序**、**中序**、**后序**、**层次**遍历也自然地可以推广至前者。

- B-树的这些遍历分别需要使用多少**内存**空间？
- 若限制只能使用 $O(1)$ 内存空间，这些遍历分别需做多少次I/O？

19. Vector::search In BTree::search

08B4

示例代码中的BTree::search()算法对各BTreeNode的查找，都是直接调用了**向量**统一的查找接口，而该接口采用的通常都是**二分查找**算法。正如课上指出的，常规阶次的BTreeNode更宜采用朴素的**顺序查找**。

- 试调整此处的代码，更改为**顺序查找**算法；
- 通过实测及统计对比，**验证**或**推翻**课上给出的这一建议。

20. Splitting An Even-Order BTreeNode

08B5

我们在课上展示的是一棵**奇数**（7）阶B-树。

- 试确认，solveOverflow()算法中的分裂策略，对**偶数**阶的B-树同样适用；
- 试确认，尽管此时的**中位数**关键码有两个，但无论取选用何者**都是**可行的。

21. Worst Case Of Splits

08B5

- 试举例说明，在**最坏**情况下，B-树的一次插入操作确实可能引发 $\Omega(\log n)$ 次分裂；
- 在**持续**插入操作的过程中，发生这种情况的**概率**有多大？
- 从**均摊**的角度来看，每次插入操作会引发多少次分裂？

22. BTree::solveOverflow()

08B5

上溢与下溢本是**对称**的缺陷，故就其原理而言，二者的修复方法及过程也应**互逆**。事实上不难确认，在修复上溢缺陷时除了**分裂**，也完全可以优先考虑**旋转**：只要不致引发兄弟节点的上溢，便向其**出让**一个关键码。

- 试改写BTree::solveOverflow()接口，加入这种策略，已确认上述**理论**上的分析确实成立；
- 在**实际**的系统实现中，这一对称的策略因何**并未**被普遍采用？

23. B*-Tree

08B5

B*-Tree是对B-树的改进版本，它采用了一种**联合分裂**的技巧：上溢的节点不是独自地分裂，而是由邻近的 k 个兄弟来共同均摊溢出的关键码，得到的 $k+1$ 个节点各自至少含有 $\lfloor (m-1) \cdot k / (k+1) \rfloor$ 个关键码，从而将空

间使用率从50%提高至 $k/(k+1)$ 。

试查阅相关资料**了解**这一改进策略，并就其效果做一**评估**。

24. Removing Keys In An Internal BTreeNode

08B6

当待删除关键码位于内部节点中时，我们需要仿照BST的策略，先将其与直接**后继**或**前驱**交换。

试验证，此时的直接后继与前驱，必然都属于某个**叶节点**。

25. No Rotates But Merge

08B6

a) solveUnderflow()算法可否如solveOverflow()那样抛弃**旋转**策略，只是一味地通过**合并**来修复下溢？

b) 如我们所见，solveUnderflow()算法只有在**无法**旋转时，才会**不情愿**地做合并。

是否可以**颠倒**这两种策略的优先级——也就是说，只要可行，就总是**优先**通过合并来修复下溢？为什么？

26. Left/Right Rotate

08B6

课上指出，solveUnderflow()中的旋转调整无论左右，均只需**常数**时间。

如果照示例代码那样，BTreeNode的key和child都系用**Vector**来实现，那么左旋、右旋对应的**常数**各是多少？

27. Direction Of Merge

08B6

不难理解，solveUnderflow()中的**合并**情况 (#2)，视下溢节点在**左**或在**右**，其实可以细分为两种情况。

然而在示例代码中，却是统一地采取了“**右**兄弟归入**左**兄弟”的策略。同时也不难理解，无论左或右，下溢节点总是要比兄弟更“**瘦**”。那么，为何不采取“**下溢**节点归入其**兄弟**”的策略，以求移动的**关键码**更少？

28. Worst Case Of Merges

08B6

a) 试举例说明，在最坏情况下，B-树的一次**删除**操作可能引发 $\Omega(\log n)$ 次**合并**；

b) 在**持续**删除操作的过程中，发生这种情况的**概率**有多大？

c) 从**均摊**的角度来看，每次删除操作会引发多少次合并？

29. Fix Capacity

08B[5+6]

我们看到，B-树每个节点中key、child向量的长度，始终都是介于**半载**与**满载**之间。

a) 试由此确认：只要在创建BTreeNode时将两个向量的容量**固定**为m，则此后它们就**无需**扩容或缩容。

b) 若Vector::shrink()将装填因子下限设定为**50%**，而不是**25%**呢？

30. Concurrency

08C1

除了课上列举的实例，你还见过哪些场合属于**并发**计算？

31. Persistence

08C1

除了课上列举的实例，你还见过哪些数据结构是**持久**的？

32. Persistence vs. Storage

08C1

设如示例代码那样，模拟并测试树形数据结构的演化过程，并用**纯文本**形式记录其在各次操作之后的快照。

a) 如果测试的规模相等，那么以下各种树形结构对应快照记录文件的大小是否会有较大**差异**：

常规BST、AVL树、伸展树、B-树、红黑树，以及第12章将介绍的完全二叉堆、锦标赛树、左式堆

b) 如果用winzip对快照记录文件做**压缩**，那么按压缩文件的大小，它们会如何**排序**？

c) 通过实测，**验证**你的估计。

33. Code Reading 08C1

红黑树已被实现并集成到多种**语言**和**系统**之中，试选择其中的两种，阅读并理解对应的代码。

34. Definition Of Red-Black Tree 08C2

讲义中定义的红黑树，是对**节点**做染色。实际上，也可以对树中的**边**做染色。

试查阅相关文献，了解其它的定义方式，并确认它们在实质上都是彼此**等效**的。

35. Imaginary External Nodes 08C2

讲义建议我们，在理解和实现红黑树的过程中，不妨**假想地**为红黑树补充足够多的外部节点，从而将其转化为一棵**真**二叉树。在后续的双红修复、双黑修复过程中，试体会这种**虚拟实验**式的想象有何妙用。

36. updateHeight() 08C2

按照红黑树的定义，任何节点x都应满足： $\text{HeightL}(x) == \text{HeightR}(x)$ 。

但是，updateHeight()为何还需要取二者之间的**大者**：

```
height = max( HeightL(this), HeightR(this) )
```

37. Minimum #Nodes 08C2

- 如果一棵红黑树的**高度**（常规定义的高度而非**黑高度**）为2，那么它**至少**包含多少个节点？
- 高度**为3呢？
- 一般地，如果将 $C(h)$ 定义为“**高度**为 h 的红黑树包含节点的最少数目”，试推导出 $C(h)$ ；
- 试证明：将 $\{1, 2, 3, \dots, C(h)\}$ 按单调**递增**的次序逐个插入，就能构造出这样的一棵红黑树；
- 颠倒过来，按单调**递减**的次序呢？
- 还有什么插入次序，可以达到**同样**的效果？

38. Initialized As Black 08C3

在按照常规BST的算法将一个节点接入红黑树之后，我们建议随即将其**染红**。当然，反过来首先**染黑**必然会违背第四条规则（所有外部节点之**黑深度**统一），但即便如此，是否同样可以**简明**地予以修复呢？

39. Uncle 08C3

在示例代码中，节点x的叔叔定义为：

```
#define Uncle( x ) ( Sibling( (x)->parent ) )
```

- 试验证，新接入的节点x如果造成双红且属于RR-1类型，则Uncle(x)必是一个（黑的）**外部节点**；
- 试验证，新接入的节点x如果造成双红且属于RR-2类型，则Uncle(x)必是一个（红的）**叶子节点**；
- 既如此，为何在修复双红缺陷的算法中，我们需要将Uncle(x)视作**一般性**的节点？

40. TallerChild() 08C3

我们看到，在RR-1情况下需要调用rotate(g)来修复双红缺陷。然而你应该记得，rotate()算法需要处理TallerChild(p)的**歧义**：p的左、右孩子可能黑高度**相等** ($\text{Height}(p \rightarrow lc) == \text{Height}(p \rightarrow rc)$)。

试确认：solveDoubleRed()在此情况下对rotate()的调用，**不会出现**上述歧义。

41. All Cases Of Double-Red 08C3

无论RR-1或RR-2，出现双红缺陷的局部都应各有**四种**情况，而讲义则只是各取了**其一**，并声称其它情况均由对称性**不难**类似地修复。

- a) 试针对讲义中**未予覆盖**的其它情况，逐一验证修复算法的**确**可行；
- b) 试对照示例代码，确认所有情况**均已完整覆盖**。

42. $\Omega(\log n)$ Recolorings

08C3

在修复双红缺陷的过程中，我们看到有可能在**每一层**都需要重染色，累计可能达到 $\Omega(\log n)$ 次。

那么从**均摊**的角度来估计，需要做多少次呢？

43. Persistence (Again)

08C3

讲义中指出，红黑树可以用来实现**持久**的BBST：因为每次插入/删除之后只需常数**次旋转**，红黑树的拓扑**结构**只有常数量级的变化，故每个新版本的**空间**复杂度得以控制在 $\mathcal{O}(1)$ 。

然而我们注意到，插入操作导致的重染色，**并不能**保证总是常数**次**，极端情况下**甚至**有可能多达 $\Omega(\log n)$ 次。

自然地，为**记录**这些颜色变化，新版本**也应**需要如此之大的空间——这部分空间消耗，为何**不予**考虑呢？

44. Imaginary Child

08C4

在讲解**双黑修复**算法时我们建议你，**假想地**认为在节点x按照常规BST的算法被删除之前，除了有一棵**子树r**作为删除之后的替代者，还有一棵**子树k**也与x同时地被删除。

- a) 试验证，此时的k其实无非就是一个**外部节点** (nullptr) ——当然，确实可以视作随同x一并被摘除；
- b) 既如此，为何在理解**整个**调整算法时，我们需要将k视作为一棵**常规**的子树——尽管它**从未真地**存在过？

45. Sibling

08C4

`solveDoubleBlack()`的每一步迭代，都要首先如此确定_r的**兄弟s**：

```
BinNodePosi<T> s = (_r == _p->lc) ? _p->rc : _p->lc;
```

- a) 这一句可否**替换**为

```
BinNodePosi<T> s = Sibling(_r)
```

- b) 通过实测，**验证**你的结论及理由。

46. TallerChild()

08C4

我们看到，`solveDoubleBlack()`在BB-1和BB-3情况下都会调用`roate(g)`。然而`rotate()`算法需要处理**TallerChild(p)**的**歧义**：p的左、右孩子可能黑高度**相等** (`Height(p->lc) == Height(p->rc)`)。

- a) BB-1对`rotate()`的调用，**是否可能**出现上述歧义？
- b) 如果**不可能**，为什么？如果**可能**，`rotate()`处理歧义的原则为什么适用于这种情况？
- c) BB-3呢？

47. All Cases Of Double-Black

08C4

双黑缺陷的四种情况各自都有多种**对称**的子情况，而讲义为简洁起见，只是各取了其中**一例**来做讲解。

- a) 试针对讲义中尚未覆盖的其它情况，逐一验证修复算法的**确**可行；
- b) 试对照示例代码，确认所有情况**均已完整覆盖**。

48. Decoupling BB-3 from BB-2B

08C4

讲义指出，尽管BB-3并不能一蹴而就**地**修复，但必然可以转化为BB-1或BB-2R。是的，BB-3不致转化为BB-2B，可以说是一种**幸运**。试设想一下，倘若二者之间不能如此地**解耦合**，将会有何**后果**？

49. Destructors**08D1**

阅读示例代码，了解Quadlist、Skiplist的析构**过程及原理**。

50. Geometric Distribution**08D1**

本节指出，按照W. Pugh的约定，Skiplist中各塔的高度服从**几何分布**，每座塔的**期望高度**均为 $\mathcal{O}(1)$ 。
试动手推算或查阅资料以确认，塔高的**方差**是多少？

51. Horizontal Hops**08D2**

本节的**重点**在于证明：Skiplist::search()的过程中，沿每一层**横向**列表都只会**期望**地跳转 $\mathcal{O}(1)$ 步。这一结论之所以成立，**关键**在于讲义中已指出的如下事实：每一步水平跳转的**目的地**，都是某座塔的**顶部**。

- a) 试**证明**这一事实；
- b) 基于该事实进一步**证明**：沿任一横向列表跳转的步数，亦符合**几何分布**。

52. Failed Trials**08D2**

Skiplist::search()沿着每一层**横向**列表的跳转，最终都会以一次**失败**的尝试（包含对**哨兵** $+\infty$ 的尝试）而结束。而在分析该算法的时间复杂度时，我们为何**似乎没有**计量这部分时间消耗？

53. Sentinels**08D3**

在Skiplist的示例代码中，每层横向列表都配置了一对**哨兵**（ $-\infty$ 和 $+\infty$ ）。
得益于这些哨兵，算法的实现在哪些方面变得更加**简捷**了？